

李良顺答辩闭环：语义分析与符号表

1. 先记住自己的定位

你这部分的核心不是“把整个编译器讲一遍”，而是守住两件事：

1. 符号表怎么存、怎么查、怎么配合作用域。
2. 语义分析怎样基于 AST 和符号表做类型检查、参数检查、语句检查。

最短自我介绍可以直接说：

我主要参与的是语义分析与符号表这一层。输入是 parser 构造好的 AST，输出是合法性检查结果；如果程序有未声明标识符、类型不匹配、参数错误或字段错误，就在这一层报出来，并阻止后续代码生成。

2. 现场先打开哪些文件

建议只记这 3 个入口，够用了：

1. [src/symbol_table.h](#)
2. [src/symbol_table.cpp](#)
3. [src/semantic_analyzer.cpp](#)

打开顺序建议也是这个顺序：

1. 先看 `SymbolEntry` 和 `SymbolTable`，讲“存什么、怎么组织”。
2. 再看 `insert` / `lookup` / `enter_scope` / `exit_scope`，讲“怎么插入、怎么查找、怎么定位”。
3. 最后看 `visit(ProgramNode)`、`visit(AssignmentNode)`、`visit(ProcedureCallNode)`，讲“语义分析到底检查了什么”。

3. 老师最可能问什么

问题 A：符号表里存了什么？结构是什么？

先打开：

- [src/symbol_table.h:11](#)
- [src/symbol_table.h:42](#)

回答模板：

我们把每一处声明都抽象成一个 `SymbolEntry`。里面保存名字、数据类型、作用域层次、位置属性 `offset`，还会按需要带上数组信息、record 字段信息、参数列表和返回类型。
整个符号表 `SymbolTable` 采用作用域栈结构，`scopes` 是一个作用域数组，每一层都是名字到符号条目的映射。进入函数或过程时压栈，退出时弹栈，所以局部变量和全局变量可以自然区分。

最好顺手指这几个字段：

- `name`

- `type`
- `scope_level`
- `offset`
- `array_info`
- `record_info`
- `params`
- `return_type`

问题 B：符号表上的操作有哪些？怎么查找、插入、定位？

先打开：

- [src/symbol_table.cpp:10](#)
- [src/symbol_table.cpp:26](#)
- [src/symbol_table.cpp:41](#)

回答模板：

插入操作对应 `insert()`，只向当前作用域写入，同时做当前层判重；如果当前作用域已经有同名符号，就返回失败，语义分析阶段据此报重复声明。

查找操作对应 `lookup()`，实现是从当前作用域往外层逐层查找，所以局部定义会优先屏蔽外层同名标识符。

定位这件事体现在 `scope_level` 和 `offset` 上：`scope_level` 表示它属于哪一层作用域，`offset` 表示它在当前作用域中的相对位置属性。

如果老师追问“为什么从里往外查”：

因为 Pascal-S 支持局部作用域，最近作用域优先才能正确实现名字屏蔽规则。

问题 C：重定位怎么处理？

这个是最容易答歪的点。

先看：

- [src/symbol_table.h:19](#)
- [src/symbol_table.cpp:10](#)

标准答法：

严格来说，本项目没有做机器级重定位。这里的 `offset` 只是符号在当前作用域里的相对位置属性，便于语义层面描述和后续阶段使用。因为我们的编译器输出的是 C 源码，不直接生成机器代码，所以真正的编译、链接和重定位都交给后续 C 编译器完成。

注意：

- 不要主动展开讲链接器原理。
- 不要说“我们实现了重定位”。
- 只要把“这里的 `offset` 不是机器级重定位”说清楚就够了。

问题 D：语义分析为什么要独立出来？

先打开：

- [src/semantic_analyzer.cpp:478](#)

回答模板：

parser 主要负责把 token 归约成 AST，如果把声明绑定和类型检查也混进去，规则会很乱。我们现在把语义分析独立成 `SemanticAnalyzer`，它在 AST 上统一做声明检查、作用域管理和类型检查。这样结构更清楚，后续扩展 `record`、`repeat-until`、错误诊断时也更稳定。

问题 E：语义分析支持哪些类型？实现了哪些检查？

先打开：

- [src/symbol_table.h:81](#)
- [src/semantic_analyzer.cpp:137](#)
- [src/semantic_analyzer.cpp:248](#)
- [src/semantic_analyzer.cpp:362](#)

标准答法：

当前支持的基础类型包括 `integer`、`real`、`boolean`、`char`，另外还支持数组、`record`、函数和过程参数。

语义分析实现的检查包括：未声明标识符、重复声明、赋值类型不匹配、表达式运算类型检查、数组下标类型检查、`record` 字段存在性检查、参数数量和类型检查、引用参数左值检查，以及 `if/while/repeat-until` 这些语句的条件合法性检查。

4. 最该会指给老师看的 5 段代码

4.1 `visit(ProgramNode)`：语义分析总入口

位置：

- [src/semantic_analyzer.cpp:478](#)

怎么讲：

这段代码先注册所有函数/过程头，再检查普通声明，再检查函数体，最后检查主程序体。
先注册子程序头的好处是，即使后面函数体里出现前向调用，名字也已经在符号表里可见。

这段是“宏观流程”，最值钱。

4.2 `visit(VariableDeclarationNode)`: 声明进入符号表

位置:

- [src/semantic_analyzer.cpp:425](#)

怎么讲:

变量声明先调用 `sym_table.insert()`, 如果失败就说明当前作用域重复声明。插入成功后, 再补充 record 信息和初始化表达式的类型检查。

老师问“插入在哪做”时, 就翻这里。

4.3 `visit(AssignmentNode)`: 赋值类型检查

位置:

- [src/semantic_analyzer.cpp:248](#)

怎么讲:

赋值语句先检查左右两边子表达式, 再确认左侧是不是可赋值的左值, 然后分别求左右类型, 最后调用类型兼容规则检查能不能赋值。

如果左边是常量, 也会在这里拦住。

这是最常见、最好讲的一段。

4.4 `visit(ProcedureCallNode)`: 参数检查

位置:

- [src/semantic_analyzer.cpp:362](#)

怎么讲:

过程调用先检查名字是否存在, 再检查它是否可调用, 然后比较实参数量和形参数量。对每个参数, 再检查类型兼容性; 如果是 `var` 引用参数, 还要额外检查实参是不是左值。

老师问“参数错误怎么发现”, 翻这一段最稳。

4.5 `visit(RecordAccessNode)`: record 字段检查

位置:

- [src/semantic_analyzer.cpp:120](#)

怎么讲:

这段先递归检查左边表达式, 然后确认它是不是 `record` 类型, 再根据字段表查找字段名。如果字段不存在, 就在这里报 `unknown record field`。

这段和你们 `caret diagnostics` 样例能直接对应起来。

5. 如果老师一个点一个点追问，怎么答

5.1 表达式检查

位置：

- [src/semantic_analyzer.cpp:137](#)

回答：

二元表达式会分别先检查左右子表达式，再看运算符种类。逻辑运算要求 `boolean`，`div/mod` 要求 `integer`，算术运算要求数值类型，比较运算则要求两侧类型兼容。

5.2 数组检查

位置：

- [src/semantic_analyzer.cpp:88](#)

回答：

数组访问会先确认名字已声明并且确实是数组，再检查维数是否匹配，最后逐个检查下标表达式是不是 `integer`。

5.3 语句检查

位置：

- [src/semantic_analyzer.cpp:289](#)
- [src/semantic_analyzer.cpp:305](#)
- [src/semantic_analyzer.cpp:318](#)

回答：

对 `if`、`while`、`repeat-until`，语义分析都会检查条件表达式是否 `boolean-compatible`。`repeat-until` 在语义上也是检查退出条件是否合法，只是代码生成阶段会把它映射成 `do ... while (!cond)`。

6. 老师如果让你“讲设计思路”，你就这么收

我们这一层的设计思路是：先让 `parser` 只负责构造 `AST`，再由 `SemanticAnalyzer` 遍历 `AST` 做合法性检查。符号表负责名字和作用域，类型系统负责兼容规则，类型查询模块负责统一推断表达式类型。这样每一层职责比较清楚，也方便后面扩展 `record`、数组和新的语句类型。

7. 最短保底版：30 秒回答

如果他非常紧张，只背这段也够过关：

我负责的是语义分析与符号表。符号表用作用域栈保存变量、常量、函数和参数信息，`lookup` 从当前作用域往外查找，`insert` 只向当前作用域插入。语义分析在 AST 上做声明绑定、类型检查和参数检查，能发现未声明标识符、重复声明、赋值类型不匹配、参数错误、数组下标错误和 record 字段错误。相关代码主要在 `symbol_table.*` 和 `semantic_analyzer.cpp`。

8. 最容易说错的 4 个点

1. 不要把 `offset` 说成机器级重定位。
正确说法：它只是当前作用域中的相对位置属性。
2. 不要说“语义分析输出 checked AST”。
你们现在更准确的说法是：语义分析输出合法性检查结果，并为后续代码生成提供符号与类型信息。
3. 不要把 `TypeResolver` 说成“又做了一遍语义分析”。
正确说法：它是统一的类型查询入口。
4. 不要把 record 检查讲成 codegen 做的。
正确说法：字段存在性在语义阶段检查，`struct` 映射在代码生成阶段完成。

9. 李良顺只需要记住的文件定位

最推荐只记下面这些：

- [src/symbol_table.h:11](#)
- [src/symbol_table.cpp:26](#)
- [src/symbol_table.cpp:41](#)
- [src/semantic_analyzer.cpp:248](#)
- [src/semantic_analyzer.cpp:362](#)
- [src/semantic_analyzer.cpp:478](#)

这 6 处能覆盖他大部分答辩问题。